University of Cape Town

Supervised Learning Assignment 2

AirBnB Bookings Prediction

Pamela Afful

Student Number: AFFPAM001

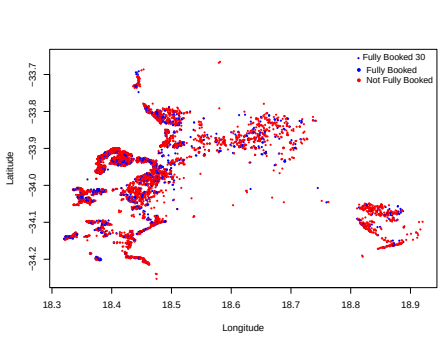
May 15, 2026

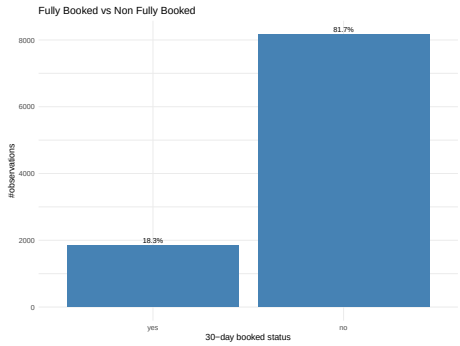
Introduction: Exploratory Data Analysis

The training data consists of 10 rows of observations across 25 features. The data is significantly imbalanced in favour of the target class at with ~81% of observations in the filly_booked_30='no' class.

Some cleaning and transforming was done on the features to make them machine learning friendly, most notably:

- Transformed all non-numerical features into factors
- Transform the **host_since_int** variable which gives the date from which the host started hostig BnBs into an interger that calculates the host's "age" in months. The feature is saved as **host_since_months_int.**

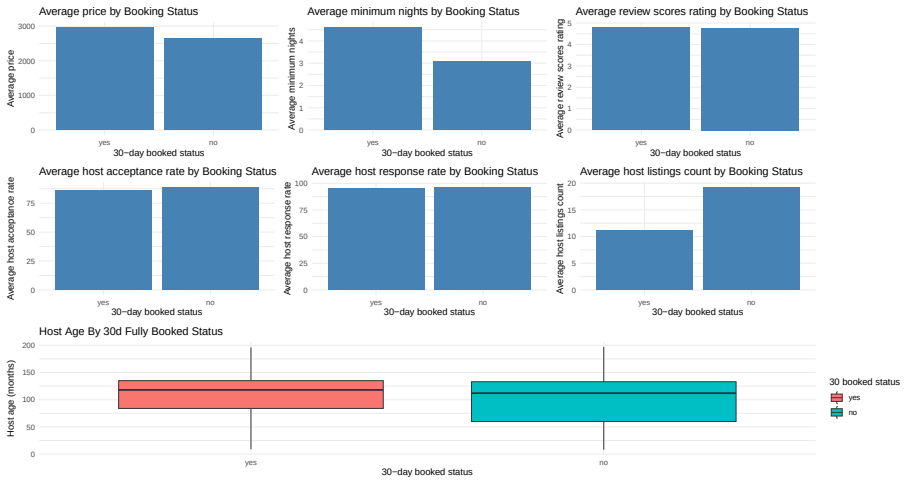




(a) Spatial Plot of BNB Listings,Fully Booked 30 Label Distribution

(a) Spatial Plot of BNB Listings,Fully Booked 30 Label Distribution

A simple decision tree to gauge which features could be important for determining the response variable. Since we have an imbalanced class, a weighted decision tree is used. The weights are a vector of ratios of the grouped observations counts of the different classes with the minority "yes" class. bieng weighted more. The ensure that the model is penalized more for misclassifying the minority class((https://machinelearningmastery.com/cost-sensitive-decision-trees-for-imbalanced-classification/))



When grouping by the target variable, fully_booked_30, the following key insights are observed:

- Average Price and average minimum nights are slightly higher for observations in the "yes" class, with the gap much wider foraverage minimum nights (~ 50 % higher than the "no" class).
- Average review scores and average response rate are fairly evenly split across the two target classes
- Average host listings are higher for the "no" class with an average of ~20 listings.
- For the host age, the median (118 months) host age is ~5% higher for the "yes" class compared to the no class. Moreover no class has a longer range of host ages, although the distributions abot the median for both groups are similar; most host ages lie below the median host age.

```
[1] yes no  no  no  no
Levels: yes no
```

A simple decision tree is ran to see how the nodes and targets are split in relation to the features

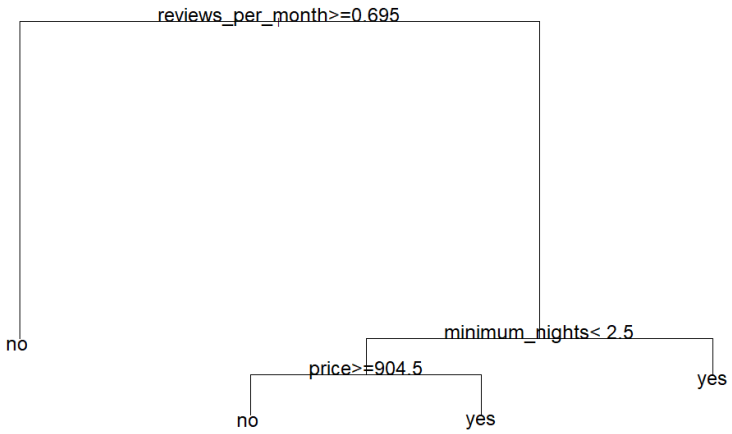


Figure 3: Initial Decision Tree Feature/ Class Splits

Features used to reduce RSS during splitting were **reviews_per_month (the largest reduction), minimum_nr_night,** and **price.**
BnB listings with reviews_per_month greater than 0.695 all had terminal nodes categorised as "no" for the response variable. Price and minimum nights were determining factors for listings with a minimum number of nights less than 2.5, which ended in "yes". Further splitting this region, for listings where the minimum was greater than 2.5, listings with prices greater than 905.5 were also categorised as "yes".

Model Training

Baseline Model Vanilla Logistic Regression

- Significant Features:
  - The number of reviews per month has the highest and significant effect on the probability of the target variable, a unit increase in the number of reviews per month increases the likelihood of an Air BnB not bieng fully booked by ~95% (Calculation e**beta, where beta=0.67).
  - The number of reviews per month has the highest and statistically significant effect on the probability of the target variable. A one-unit increase in the number of reviews per month increases the likelihood of an Airbnb not being fully booked by approximately 95%\footnote{This is calculated using $ e^{\beta}$, where $\beta = 0.67$,

Characteristic	**log(OR)**[1]	**SE**
(Intercept)	-26*	10.2
host_response_time		
within an hour	—	—
within a few hours	-0.18*	0.073
within a day	-0.42***	0.097
a few days or more	-0.47	0.324
host_response_rate	0.00	0.003
host_acceptance_rate	0.00*	0.002
host_is_superhost		
t	—	—
f	-0.07	0.060
host_listings_count	0.01***	0.001
host_verifications		
phone_only	—	—
phone_email	-0.05	0.097
email_only	-2.0	1.50
latitude	-0.56*	0.278
longitude	0.54*	0.252
room_type		
Entire home/apt	—	—
Private room	0.15	0.103
accommodates	0.01	0.030
bathrooms	0.08	0.047
bedrooms	-0.12*	0.056
beds	0.00	0.027
price	0.00	0.000
minimum_nights	0.01	0.011
maximum_nights	0.00	0.000
minimum_nights_avg_ntm	-0.02	0.011
maximum_nights_avg_ntm	0.00	0.000
number_of_reviews	0.00	0.001
reviews_per_month	0.67***	0.054
review_scores_rating	-0.23**	0.085
instant_bookable		
t	—	—
f	-0.21**	0.073
host_since_months_int	0.00***	0.001

[1]*p<0.05; **p<0.01; ***p<0.001

Abbreviations: CI = Confidence Interval, OR = Odds Ratio, SE = Standard Error

No. Obs. = 10,000

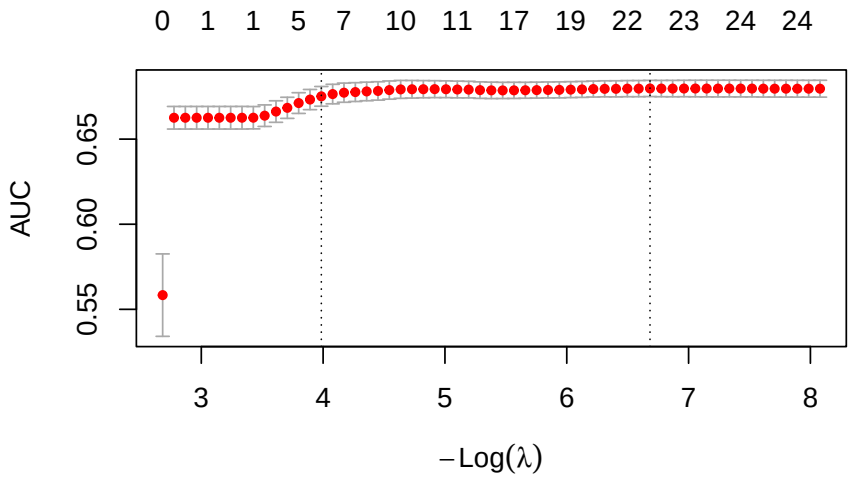
giving $e^{0.67} \approx 1.95$.}

‘

  - Latitude and Longitude- location places a significant role BnB demand and the likelihood of a BnB being fully booked- although the signs for both features are different
  - Although features host_acceptance_rate is significant, host_since_months_int and host_acceptance_rate are significant they does not seem to have an extremly negligeble effect on the likelihood of the positive class(target=‘no’).
  - Host Listings Count
  - Review Score Ratings
  - Instant bookable=F,
  - Host Since (Measured in months)
- Insignificant Features:
  - The rest* comeback
  - Most interestingly the price!-

L1 Regularization

```
[1] 0.9542373
```



From the above, the best number of non-zero coefficients equals 9 (using 1SE discrimination). This is used to identify the zero-coefficient features.

Feature	Coefficient
(Intercept)	1.1863048
host_response_timewithin a few hours	0.0000000
host_response_timewithin a day	-0.1219905
host_response_timea few days or more	0.0000000
host_response_rate	0.0000000
host_acceptance_rate	0.0000000
host_is_superhostf	0.0000000
host_listings_count	0.0022551
host_verificationsphone_email	0.0000000
host_verificationsemail_only	0.0000000
latitude	0.0000000
longitude	0.0000000
room_typePrivate room	0.0000000
accommodates	0.0000000
bathrooms	0.0000000
bedrooms	0.0000000
beds	0.0000000
price	0.0000000
minimum_nights	0.0000000
maximum_nights	0.0000000
minimum_nights_avg_ntm	-0.0022060
maximum_nights_avg_ntm	0.0000000
number_of_reviews	0.0000000
reviews_per_month	0.4268373
review_scores_rating	0.0000000
instant_bookablef	-0.0642674
host_since_months_int	-0.0004212

Penalized Features:

- response ratwe
- acceptance rate
- is_superhost
- host ver
- lat and long
- all those in red

Random Forest

Using all non_zero_coeff

```
[1] 4.452563 1.000000
```

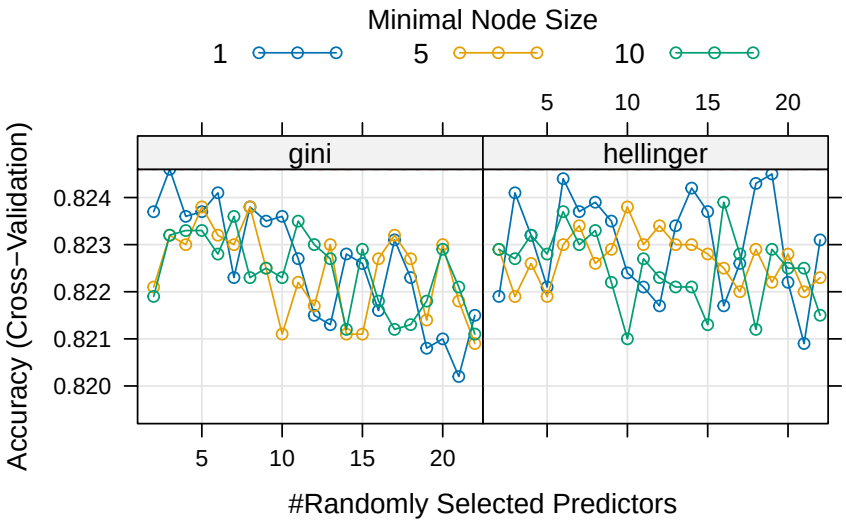


Table 1: Best RF HyperParameters

	mtry	splitrule	min.node.size
7	3	gini	1

Boosting Model

Used non-zero features from the regularized model to speed up training- we've established that the majority of these are not relevant anyway through CV and the

Used features:

```
function (package, help, pos = 2, lib.loc = NULL, character.only = FALSE,
    logical.return = FALSE, warn.conflicts, quietly = FALSE,
    verbose = getOption("verbose"), mask.ok, exclude, include.only,
    attach.required = missing(include.only))
{
    conf.ctrl <- getOption("conflicts.policy")
    if (is.character(conf.ctrl))
        conf.ctrl <- switch(conf.ctrl, strict = list(error = TRUE,
            warn = FALSE), depends.ok = list(error = TRUE, generics.ok = TRUE,
```

```
            can.mask = c("base", "methods", "utils", "grDevices",
                "graphics", "stats"), depends.ok = TRUE), warning(gettextf("unknown conflict
            sQuote(conf.ctrl)), call. = FALSE, domain = NA))
    if (!is.list(conf.ctrl))
        conf.ctrl <- NULL
    stopOnConflict <- isTRUE(conf.ctrl$error)
    if (missing(warn.conflicts))
        warn.conflicts <- !isFALSE(conf.ctrl$warn)
    if (!missing(include.only) && !missing(exclude))
        stop("only one of 'include.only' and 'exclude' can be used",
            call. = FALSE)
    testRversion <- function(pkgInfo, pkgname, pkgpath) {
        if (is.null(built <- pkgInfo$Built))
            stop(gettextf("package %s has not been installed properly\n",
                sQuote(pkgname)), call. = FALSE, domain = NA)
        R_version_built_under <- as.numeric_version(built$R)
        if (R_version_built_under < "3.0.0")
            stop(gettextf("package %s was built before R 3.0.0: please re-
install it",
                sQuote(pkgname)), call. = FALSE, domain = NA)
        current <- getRversion()
        if (length(Rdeps <- pkgInfo$Rdepends2)) {
            for (dep in Rdeps) if (length(dep) > 1L) {
                target <- dep$version
                res <- do.call(dep$op, if (is.character(target))
                  list(as.numeric(R.version[["svn rev"]]), as.numeric(sub("^r",
                    "", target)))
                else list(current, as.numeric_version(target)))
                if (!res)
                  stop(gettextf("This is R %s, package %s needs %s %s",
                    current, sQuote(pkgname), dep$op, target),
                    call. = FALSE, domain = NA)
            }
        }
        if (R_version_built_under > current)
            warning(gettextf("package %s was built under R version %s",
                sQuote(pkgname), as.character(built$R)), call. = FALSE,
                domain = NA)
        platform <- built$Platform
        r_arch <- .Platform$r_arch
        if (.Platform$OS.type == "unix") {
        }
        else {
            if (nzchar(platform) && !grepl("mingw", platform))
                stop(gettextf("package %s was built for %s",
                  sQuote(pkgname), platform), call. = FALSE,
```

```
            domain = NA)
    }
    if (nzchar(r_arch) && file.exists(file.path(pkgpath,
        "libs")) && !file.exists(file.path(pkgpath, "libs",
        r_arch)))
        stop(gettextf("package %s is not installed for 'arch = %s'",
            sQuote(pkgname), r_arch), call. = FALSE, domain = NA)
}
checkNoGenerics <- function(env, pkg) {
    nenv <- env
    ns <- .getNamespace(as.name(pkg))
    if (!is.null(ns))
        nenv <- asNamespace(ns)
    if (exists(".noGenerics", envir = nenv, inherits = FALSE))
        TRUE
    else {
        !any(startsWith(names(env), ".__T"))
    }
}
checkConflicts <- function(package, pkgname, pkgpath, nogenerics,
    env) {
    dont.mind <- c("last.dump", "last.warning", ".Last.value",
        ".Random.seed", ".Last.lib", ".onDetach", ".packageName",
        ".noGenerics", ".required", ".no_S3_generics", ".Depends",
        ".requireCachedGenerics")
    sp <- search()
    lib.pos <- which(sp == pkgname)
    ob <- names(as.environment(lib.pos))
    if (!nogenerics) {
        these <- ob[startsWith(ob, ".__T__")]
        gen <- gsub(".__T__(.*):([^:]+)", "\\1", these)
        from <- gsub(".__T__(.*):([^:]+)", "\\2", these)
        gen <- gen[from != package]
        ob <- ob[!(ob %in% gen)]
    }
    ipos <- seq_along(sp)[-c(lib.pos, match(c("Autoloads",
        "CheckExEnv"), sp, 0L))]
    cpos <- NULL
    conflicts <- vector("list", 0)
    for (i in ipos) {
        obj.same <- match(names(as.environment(i)), ob, nomatch = 0L)
        if (any(obj.same > 0L)) {
            same <- ob[obj.same]
            same <- same[!(same %in% dont.mind)]
            Classobjs <- which(startsWith(same, ".__"))
            if (length(Classobjs))
```

```
            same <- same[-Classobjs]
          same.isFn <- function(where) vapply(same, exists,
            NA, where = where, mode = "function", inherits = FALSE)
          same <- same[same.isFn(i) == same.isFn(lib.pos)]
          not.Ident <- function(ch, TRAFO = identity, ...) vapply(ch,
            function(.) !identical(TRAFO(get(., i)), TRAFO(get(.,
              lib.pos)), ...), NA)
          if (length(same))
            same <- same[not.Ident(same)]
          if (length(same) && identical(sp[i], "package:base"))
            same <- same[not.Ident(same, ignore.environment = TRUE)]
          if (length(same)) {
            conflicts[[sp[i]]] <- same
            cpos[sp[i]] <- i
          }
      }
  }
  if (length(conflicts)) {
      if (stopOnConflict) {
          emsg <- ""
          pkg <- names(conflicts)
          notOK <- vector("list", 0)
          for (i in seq_along(conflicts)) {
            pkgname <- sub("^package:", "", pkg[i])
            if (pkgname %in% canMaskEnv$canMask)
              next
            same <- conflicts[[i]]
            if (is.list(mask.ok))
              myMaskOK <- mask.ok[[pkgname]]
            else myMaskOK <- mask.ok
            if (isTRUE(myMaskOK))
              same <- NULL
            else if (is.character(myMaskOK))
              same <- setdiff(same, myMaskOK)
            if (length(same)) {
              notOK[[pkg[i]]] <- same
              msg <- .maskedMsg(sort(same), pkg = sQuote(pkg[i]),
                by = cpos[i] < lib.pos)
              emsg <- paste(emsg, msg, sep = "\n")
            }
          }
          if (length(notOK)) {
            msg <- gettextf("Conflicts attaching package %s:\n%s",
              sQuote(package), emsg)
            stop(errorCondition(msg, package = package,
              conflicts = conflicts, class = "packageConflictError"))
```

```
            }
        }
        if (warn.conflicts) {
            packageStartupMessage(gettextf("\nAttaching package: %s\n",
              sQuote(package)), domain = NA)
            pkg <- names(conflicts)
            for (i in seq_along(conflicts)) {
              msg <- .maskedMsg(sort(conflicts[[i]]), pkg = sQuote(pkg[i]),
                by = cpos[i] < lib.pos)
              packageStartupMessage(msg, domain = NA)
            }
        }
    }
}
if (verbose && quietly)
    message("'verbose' and 'quietly' are both true; being verbose then ..")
if (!missing(package)) {
    if (is.null(lib.loc))
        lib.loc <- .libPaths()
    lib.loc <- lib.loc[dir.exists(lib.loc)]
    if (!character.only)
        package <- as.character(substitute(package))
    if (length(package) != 1L)
        stop(gettextf("'%s' must be of length 1", "package"),
            domain = NA)
    if (is.na(package) || (package == ""))
        stop("invalid package name")
    pkgname <- paste0("package:", package)
    newpackage <- is.na(match(pkgname, search()))
    if (newpackage) {
        pkgpath <- find.package(package, lib.loc, quiet = TRUE,
            verbose = verbose)
        if (length(pkgpath) == 0L) {
            if (length(lib.loc) && !logical.return)
              stop(packageNotFoundError(package, lib.loc,
                sys.call()))
            txt <- if (length(lib.loc))
              gettextf("there is no package called %s", sQuote(package))
            else gettext("no library trees found in 'lib.loc'")
            if (logical.return) {
              if (!quietly)
                warning(txt, domain = NA)
              return(FALSE)
            }
            else stop(txt, domain = NA)
        }
```

```
which.lib.loc <- normalizePath(dirname(pkgpath),
    "/", TRUE)
pfile <- system.file("Meta", "package.rds", package = package,
    lib.loc = which.lib.loc)
if (!nzchar(pfile))
    stop(gettextf("%s is not a valid installed package",
      sQuote(package)), domain = NA)
pkgInfo <- readRDS(pfile)
testRversion(pkgInfo, package, pkgpath)
if (is.character(pos)) {
    npos <- match(pos, search())
    if (is.na(npos)) {
      warning(gettextf("%s not found on search path, using pos = 2",
        sQuote(pos)), domain = NA)
      pos <- 2
    }
    else pos <- npos
}
deps <- unique(names(pkgInfo$Depends))
depsOK <- isTRUE(conf.ctrl$depends.ok)
if (depsOK) {
    canMaskEnv <- dynGet("__library_can_mask__",
      NULL)
    if (is.null(canMaskEnv)) {
      canMaskEnv <- new.env()
      canMaskEnv$canMask <- union("base", conf.ctrl$can.mask)
      "__library_can_mask__" <- canMaskEnv
    }
    canMaskEnv$canMask <- unique(c(package, deps,
      canMaskEnv$canMask))
}
else canMaskEnv <- NULL
if (attach.required)
    .getRequiredPackages2(pkgInfo, quietly = quietly,
      lib.loc = c(lib.loc, .libPaths()))
cr <- conflictRules(package)
if (missing(mask.ok))
    mask.ok <- cr$mask.ok
if (missing(exclude))
    exclude <- cr$exclude
if (isNamespaceLoaded(package)) {
    newversion <- as.numeric_version(pkgInfo$DESCRIPTION["Version"])
    oldversion <- as.numeric_version(getNamespaceVersion(package))
    if (newversion != oldversion) {
      tryCatch(unloadNamespace(package), error = function(e) {
        P <- if (!is.null(cc <- conditionCall(e)))
```

```
                paste("Error in", deparse(cc)[1L], ": ")
              else "Error : "
              stop(gettextf("Package %s version %s cannot be unloaded:\n %s",
                sQuote(package), oldversion, paste0(P,
                  conditionMessage(e), "\n")), domain = NA)
            })
        }
    }
    tt <- tryCatch({
        attr(package, "LibPath") <- which.lib.loc
        ns <- loadNamespace(package, lib.loc)
        env <- attachNamespace(ns, pos = pos, deps, exclude,
          include.only)
    }, error = function(e) {
        P <- if (!is.null(cc <- conditionCall(e)))
          paste(" in", deparse(cc)[1L])
        else ""
        msg <- gettextf("package or namespace load failed for %s%s:\n %s",
          sQuote(package), P, conditionMessage(e))
        if (logical.return && !quietly)
          message(paste("Error:", msg), domain = NA)
        else stop(msg, call. = FALSE, domain = NA)
    })
    if (logical.return && is.null(tt))
        return(FALSE)
    attr(package, "LibPath") <- NULL
    {
        on.exit(detach(pos = pos))
        nogenerics <- !.isMethodsDispatchOn() || checkNoGenerics(env,
          package)
        if (isFALSE(conf.ctrl$generics.ok) || (stopOnConflict &&
          !isTRUE(conf.ctrl$generics.ok)))
          nogenerics <- TRUE
        if (stopOnConflict || (warn.conflicts && !exists(".conflicts.OK",
          envir = env, inherits = FALSE)))
          checkConflicts(package, pkgname, pkgpath, nogenerics,
            ns)
        on.exit()
        if (logical.return)
          return(TRUE)
        else return(invisible(.packages()))
    }
}
if (verbose && !newpackage)
    warning(gettextf("package %s already present in search()",
        sQuote(package)), domain = NA)
```

```
    }
    else if (!missing(help)) {
        if (!character.only)
            help <- as.character(substitute(help))
        pkgName <- help[1L]
        pkgPath <- find.package(pkgName, lib.loc, verbose = verbose)
        docFiles <- c(file.path(pkgPath, "Meta", "package.rds"),
            file.path(pkgPath, "INDEX"))
        if (file.exists(vignetteIndexRDS <- file.path(pkgPath,
            "Meta", "vignette.rds")))
            docFiles <- c(docFiles, vignetteIndexRDS)
        pkgInfo <- vector("list", 3L)
        readDocFile <- function(f) {
            if (basename(f) %in% "package.rds") {
                txt <- readRDS(f)$DESCRIPTION
                if ("Encoding" %in% names(txt)) {
                  to <- if (Sys.getlocale("LC_CTYPE") == "C")
                    "ASCII//TRANSLIT"
                  else ""
                  tmp <- try(iconv(txt, from = txt["Encoding"],
                    to = to))
                  if (!inherits(tmp, "try-error"))
                    txt <- tmp
                  else warning("'DESCRIPTION' has an 'Encoding' field and re-
encoding is not possible",
                      call. = FALSE)
                }
                nm <- paste0(names(txt), ":")
                formatDL(nm, txt, indent = max(nchar(nm, "w")) +
                  3L)
            }
            else if (basename(f) %in% "vignette.rds") {
                txt <- readRDS(f)
                if (is.data.frame(txt) && nrow(txt))
                  cbind(basename(gsub("\\.[[:alpha:]]+$", "",
                    txt$File)), paste(txt$Title, paste0(rep.int("(source",
                    NROW(txt)), ifelse(nzchar(txt$PDF), ", pdf",
                    ""), ")")))
                else NULL
            }
            else readLines(f)
        }
        for (i in which(file.exists(docFiles))) pkgInfo[[i]] <- readDocFile(docFiles[i])
        y <- list(name = pkgName, path = pkgPath, info = pkgInfo)
        class(y) <- "packageInfo"
        return(y)
```

```
    }
    else {
        if (is.null(lib.loc))
            lib.loc <- .libPaths()
        db <- matrix(character(), nrow = 0L, ncol = 3L)
        nopkgs <- character()
        for (lib in lib.loc) {
            a <- .packages(all.available = TRUE, lib.loc = lib)
            for (i in sort(a)) {
                file <- system.file("Meta", "package.rds", package = i,
                  lib.loc = lib)
                title <- if (nzchar(file)) {
                  txt <- readRDS(file)
                  if (is.list(txt))
                    txt <- txt$DESCRIPTION
                  if ("Encoding" %in% names(txt)) {
                    to <- if (Sys.getlocale("LC_CTYPE") == "C")
                      "ASCII//TRANSLIT"
                    else ""
                    tmp <- try(iconv(txt, txt["Encoding"], to,
                      "?"))
                    if (!inherits(tmp, "try-error"))
                      txt <- tmp
                    else warning("'DESCRIPTION' has an 'Encoding' field and re-
encoding is not possible",
                         call. = FALSE)
                  }
                  txt["Title"]
                }
                else NA
                if (is.na(title))
                  title <- " ** No title available ** "
                db <- rbind(db, cbind(i, lib, title))
            }
            if (length(a) == 0L)
                nopkgs <- c(nopkgs, lib)
        }
        dimnames(db) <- list(NULL, c("Package", "LibPath", "Title"))
        if (length(nopkgs) && !missing(lib.loc)) {
            pkglist <- paste(sQuote(nopkgs), collapse = ", ")
            msg <- sprintf(ngettext(length(nopkgs), "library %s contains no packages",
                "libraries %s contain no packages"), pkglist)
            warning(msg, domain = NA)
        }
        y <- list(header = NULL, results = db, footer = NULL)
        class(y) <- "libraryIQR"
```

```
        return(y)
    }
    if (logical.return)
        TRUE
    else invisible(.packages())
}
<bytecode: 0x0000026160170b88>
<environment: namespace:base>
```

```
plot(gbm_gridsearch)
```

The best parameter is acquired at the hyperparameter combinations below:

Table 2: GBM Best HyperParameter Combination

	nr trees	interaction depth	shrinkage
60	5000	9	0.1

```
# using all features comparability

library(doParallel)
library(foreach)
set.seed(123)

# ctrl = trainControl(method = 'cv', number = 10, verboseIter = F,

ctrl = trainControl(method = 'cv', number = 10, verboseIter = F,


gbm_grid <- expand.grid(n.trees = c(500,1000,1500),
                                interaction.depth = seq(1, 5, 1),
                                shrinkage =c(0.005, 0.01),
                                n.minobsinnode = 10)

#CV for hyper parameter tuning
# n_cores <- parallel::detectCores() - 1  # leave one core free
# cl <- makeCluster(n_cores)
# clusterEvalQ(cl, library(Metrics))
# registerDoParallel(cl)
# clusterExport(cl, varlist = c("log_loss"))
#
# gbm_all_gridsearch <- train(fully_booked_30 ~ .,data =listings_data_cleaned_features,
#                                 method = 'gbm',
#                                 distribution = "bernoulli", #
```

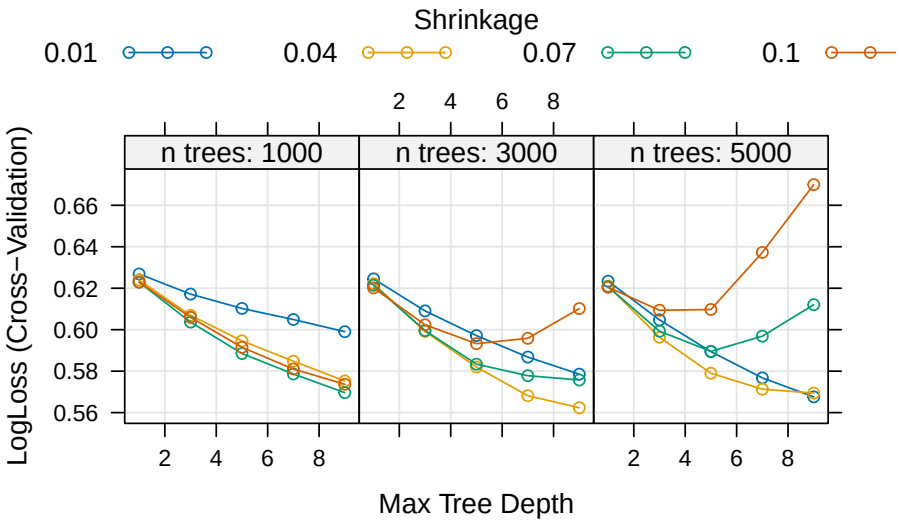
```
#                                 trControl = ctrl,
#                                 verbose = T,
#                                 tuneGrid = gbm_grid,
#                                 metric = "LogLoss",
#                                 weights=weights_vector)
#
# stopCluster(cl)
# registerDoSEQ()


#save(gbm_all_gridsearch , file = 'gbm_all_gridsearch.Rdata')

saveRDS(rf_gridsearch, file = "caret_rf_grid_results_cv_weights.rds")

#saveRDS(rf_gridsearch, file = "caret_rf_grid_results_cv_weights.rds")


#load('gbm_all_gridsearch.Rdata')
```




```
kable(gbm_gridsearch$bestTune %>% dplyr:: select(n.trees,interaction.depth,shrinkage) %>%
        rename(`nr trees`=n.trees,`interaction depth`=interaction.depth)
        , caption = 'GBM Best HyperParameter Combination')
```

Table 3: GBM Best HyperParameter Combination

	nr trees	interaction depth	shrinkage
60	5000	9	0.1

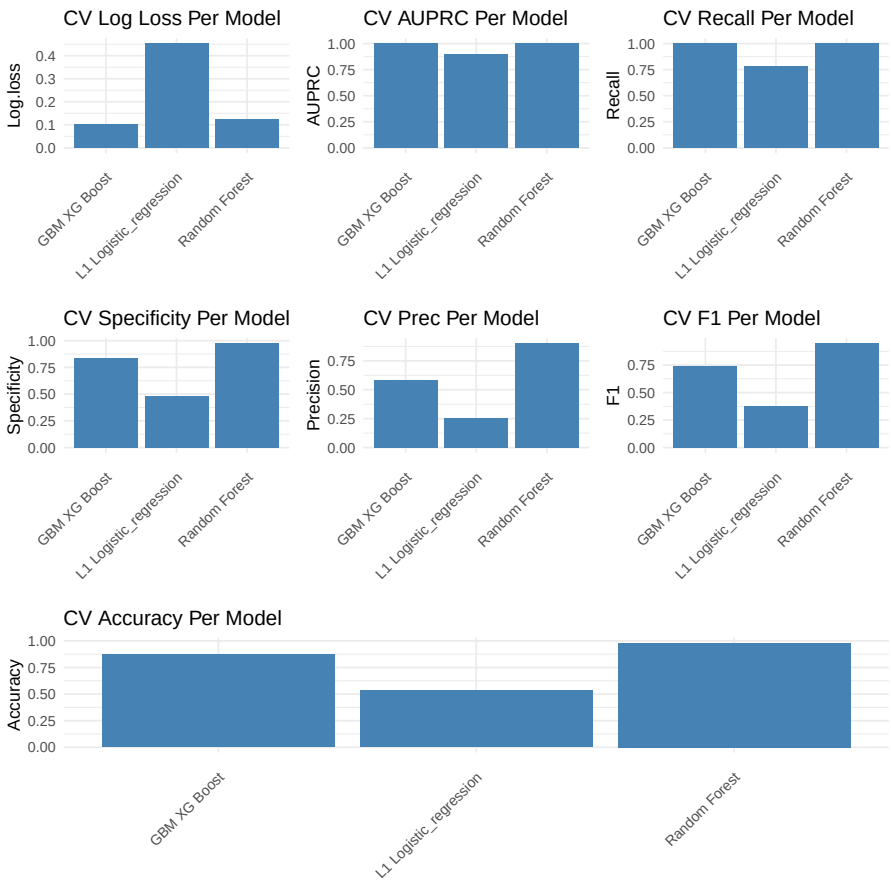
Question 3- Model Evaluation and Selection

a)

```
[1] 0.8166
```

```
                   model  Log.loss     AUPRC    Recall Specificity Precision
1 L1 Logistic_regression 0.4525573 0.8996259 0.7770254   0.4796749 0.2512258
         F1  Accuracy
1 0.3796695 0.5342054
```

```
         model  Log.loss     AUPRC Recall Specificity Precision        F1
1 GBM XG Boost 0.1035011 0.9998888      1   0.8408813 0.5846645 0.7379032
  Accuracy
1     0.87
```

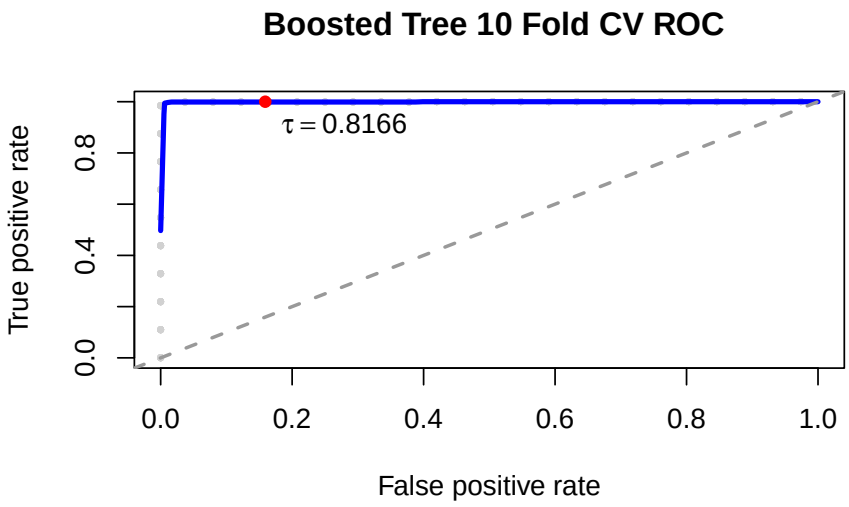


Boosting tree is the best model, given all the metrics:

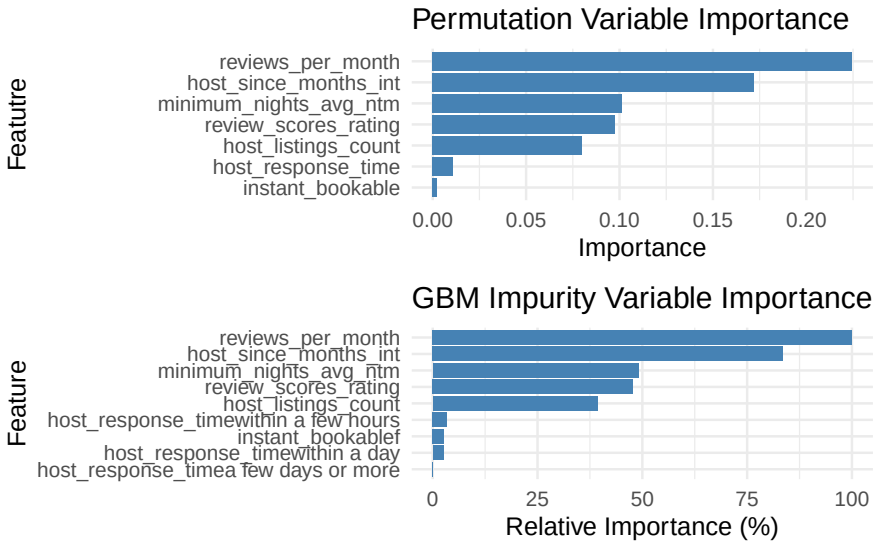
- Lowest penalty of negative predictions from log loss- primary metric
- Best AUPRC- best metric for imbalanced data set

```
[1] 1
```

```
[1] 0.1591187
```



Section 4- Interpretation



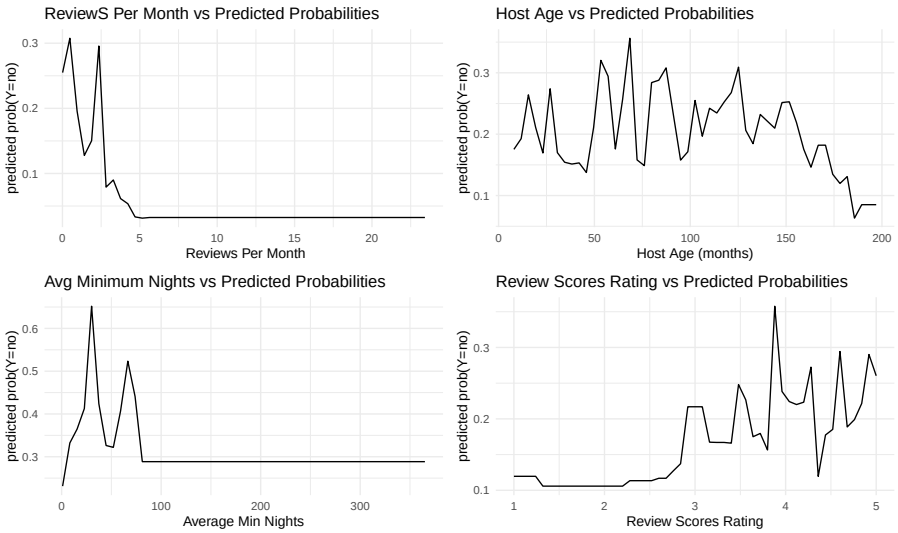


Figure 4: Partial Dependence Plots for Top 4 Features